ECE 243 - Final Report
Tej Patel and Leo Zou
Safe Cracker

1.0 Overview

Safe Cracker is a timed and lives-based game for the DE1-SoC that simulates opening a rotational safe lock with a rotary encoder. It combines VGA, a rotary encoder, KEYs, HEX displays, LEDs, SWs, and audio. The safe lock is shown on screen and responds to the user's input. Before starting, the user may choose a time limit and the number of stages to guess. The user spins the rotary encoder to find the hidden random sequence that opens the lock.

To crack the safe, the user must alternate the rotation direction for each stage in a clockwise, counterclockwise, clockwise pattern like a real lock. The user must also listen for unique audio that indicates the correct number for the current stage. The user has 3 guesses and presses KEY0 to confirm each guess. If the user fails to crack the code within 3 guesses or before time expires, the game enters a fail screen and plays an alarm. If the full sequence is entered correctly, the game enters a pass screen. During gameplay, unique sounds play when turning the rotary encoder, landing on the correct number for the current stage, on correct/wrong guesses, when the lock is opened, and when the user fails to unlock it.

2.0 Operation

Below is the general operation for Safe Cracker, with inputs described per phase of the game.

2.1 Setup

- SW0-9: Choose how many numbers to guess (stages) in the sequence (minimum 1, maximum 10)
- KEY3: Decrease the amount of time (minimum 7 seconds, decrease in 10s increments)
- KEY2: Increase the amount of time (maximum 670s, increase in 10s increments)
- KEY1: Enter/exit help screen
- KEY0: Confirm settings and start game
- HEX0-1: Displays the number of stages to guess

2.2 Gameplay

- Rotary Encoder: Spin to find the correct number. Alternate spin direction after each stage (full on-screen encoder rotation is 0-39, 2 full physical encoder rotations)
- KEY0: Confirm guess, exit pass and fail screens
- HEX0-1: Dial position (0-39)
- HEX2-3: Stage the user is currently on
- HEX4-5: Number of guesses that are remaining
- LEDR0-9: Current stage progress indication

3.0 Block Diagram

The game flow of Safe Cracker is shown below in Figure 1.

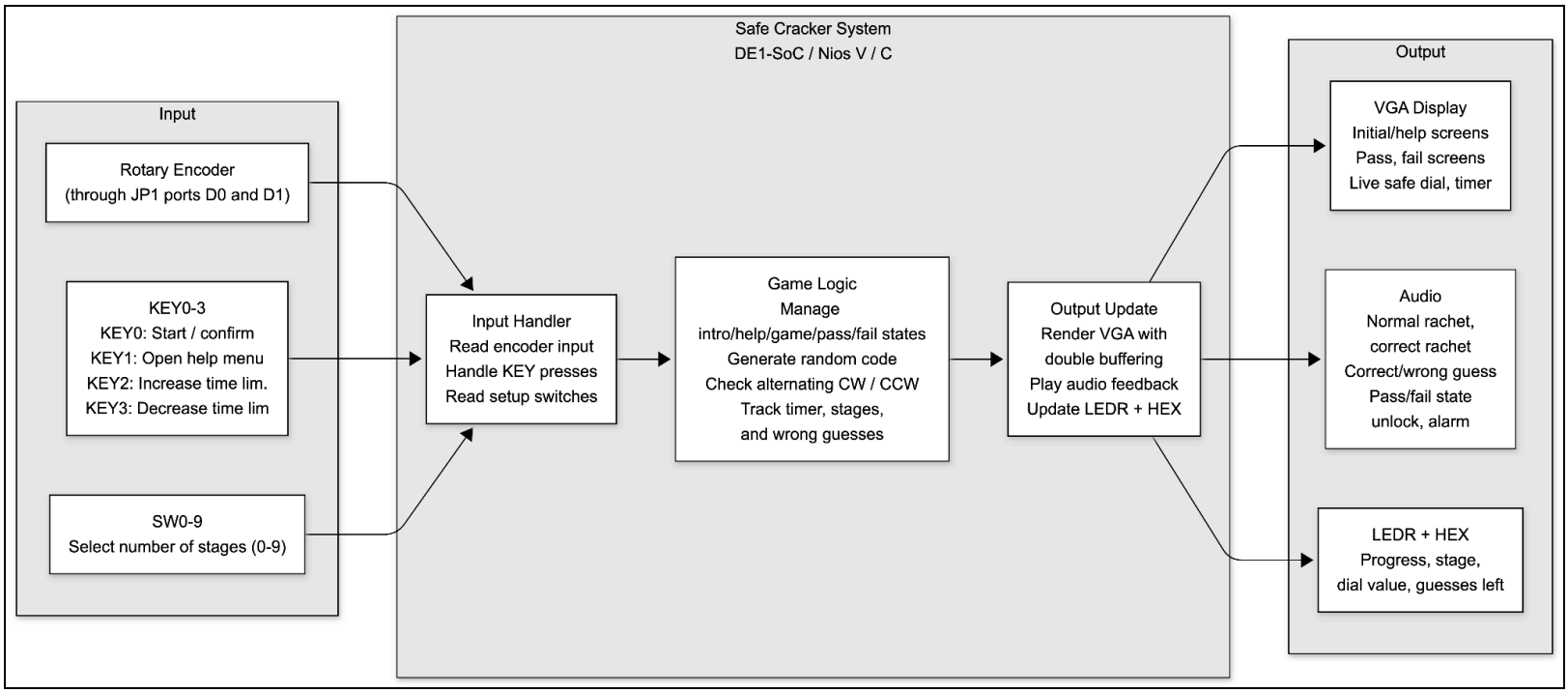

Figure 1. Block Diagram of Safe Cracker

4.0 Attribution Table

Below is a summary of the work distribution between Tej and Leo. The specific contributions and implementations are explained in Table 1.

Table 1. Work Distribution

Member	Project Contribution	Explanation
Tej Patel	Rotary Encoder	Connected rotary encoder to JP1 expansion ports, and used interrupts to detect input, track full rotation, clockwise and counterclockwise directions. Used machine timer to reduce debounce with delay.
	KEYs	Enabled KEYs to enter help screen, enter game screen, confirm answer, and exit fail/pass screens. Used interrupts to detect input.
	HEX Displays + LEDs	Displayed various game parameters on the hex displays based on the game phase. Made LEDs light up based on stage.
	SWs	Allowed SWs to set the number of numbers that the user must guess during gameplay. Each SW0-9 increases the number of stages by 1.

	Audio	Added and implemented all audio. Converted MP3 files to C arrays and created functions to set up, load, and clear audio fifos, with blocking and non-blocking audio.
	Game Logic + Combining VGA and game logic	Created most game logic, such as random code generation, scoring, guess attempts, and merged game logic and VGA functionality, allowing hardware to control the corresponding elements on the screen.
Leo Zou	VGA Visuals System	Set up the full VGA graphics system with drawing by pixel and double buffering. Wrote all the main visual functions for drawing lines and circles, updating the frame buffer with wait_for_vsync.
	Live Dial Logic + Display	Designed and implemented the live safe dial visuals, including the rotating numbered dial, red pointer, long/short tick marks, and center knob.
	Timer Logic	Configured the FPGA interval timer to interrupt once per second for timer decrease and enter the fail state when the timer = 0. Implemented the time selection controls using KEY2 to increase the time limit and KEY3 to decrease.
	Timer Display	Wrote the digit drawing functions used to render a 3-digit timer. Integrated the timer display on the top right corner of the VGA display.
	Game Screens Design + Integration	Created and integrated the intro, help, gameplay, pass, fail, and wall background screens. Handled the placement and rendering of the safe, wall, and pass/fail emojis in each state.